【转载】第六讲——使用Frida进行DFA攻击

转载 (https://www.qinless.com/category/reprint) · 2022-04-01 · 466 次浏览

文章目录[隐藏]

- 转载 23 / 25
- 说说在前面
- 一、前言
- 二、案例
- 三、关于自动化的思考
- 四、尾声

转载 (https://www.qinless.com/series/zhuanzai) 23 / 25

【转载】unidbg hook 使用大全 (https://www.qinless.com/670)

【转载】ida 基本使用 常用快捷键 (https://www.qinless.com/732)

【转载】Frida和调试器共存的问题 (https://www.qinless.com/937)

【转载】Charles抓包Android最佳流量转发APP (https://www.qinless.com/939)

【转载】安卓自定义代码调用系统函数并编译so记录 (https://www.qinless.com/1020)

【转载】jnitrace 手机卡死黑屏问题解决 (https://www.qinless.com/1169)

【转载】Unidbg SO 逆向入门实战教程一 OASIS (https://www.qinless.com/1173)

【转载】Unidbg SO 逆向入门实战教程二 calculateS (https://www.qinless.com/1177)

【转载】Unidbg SO 逆向入门实战教程三 V2-Sign (https://www.qinless.com/1181)

【转载】Unidbg SO 逆向入门实战教程四 mfw (https://www.qinless.com/1185)

【转载】Unidbg SO 逆向入门实战教程五 qxs (https://www.qinless.com/1188)

【转载】Unidbg SO 逆向入门实战教程六 s (https://www.qinless.com/1192)

【转载】Unidbg SO 逆向入门实战教程七 main (https://www.qinless.com/1194)

【转载】Unidbg SO 逆向入门实战教程八 文件读写 (https://www.qinless.com/1201)

【转载】Unidbg SO 逆向入门实战教程九 blackbox (https://www.qinless.com/1204)

【转载】Unidbg SO 逆向入门实战教程十 simpleSign (https://www.qinless.com/1209)

【转载】android app 加密参数分析研究 hash aes (https://www.qinless.com/1634)

【转载】第一讲——从黑盒攻击模型到白盒攻击模型 (https://www.qinless.com/1642)

【转载】第二讲——白盒加密攻击方法的选择 (https://www.qinless.com/1645)

【转载】第三讲——差分故障攻击的原理 (https://www.qinless.com/1647)

【转载】第四讲——差分故障攻击的工具链 (https://www.qinless.com/1653)

【转载】第五讲——使用源码进行DFA攻击 (https://www.qinless.com/1655)

【转载】第六讲——使用Frida进行DFA攻击

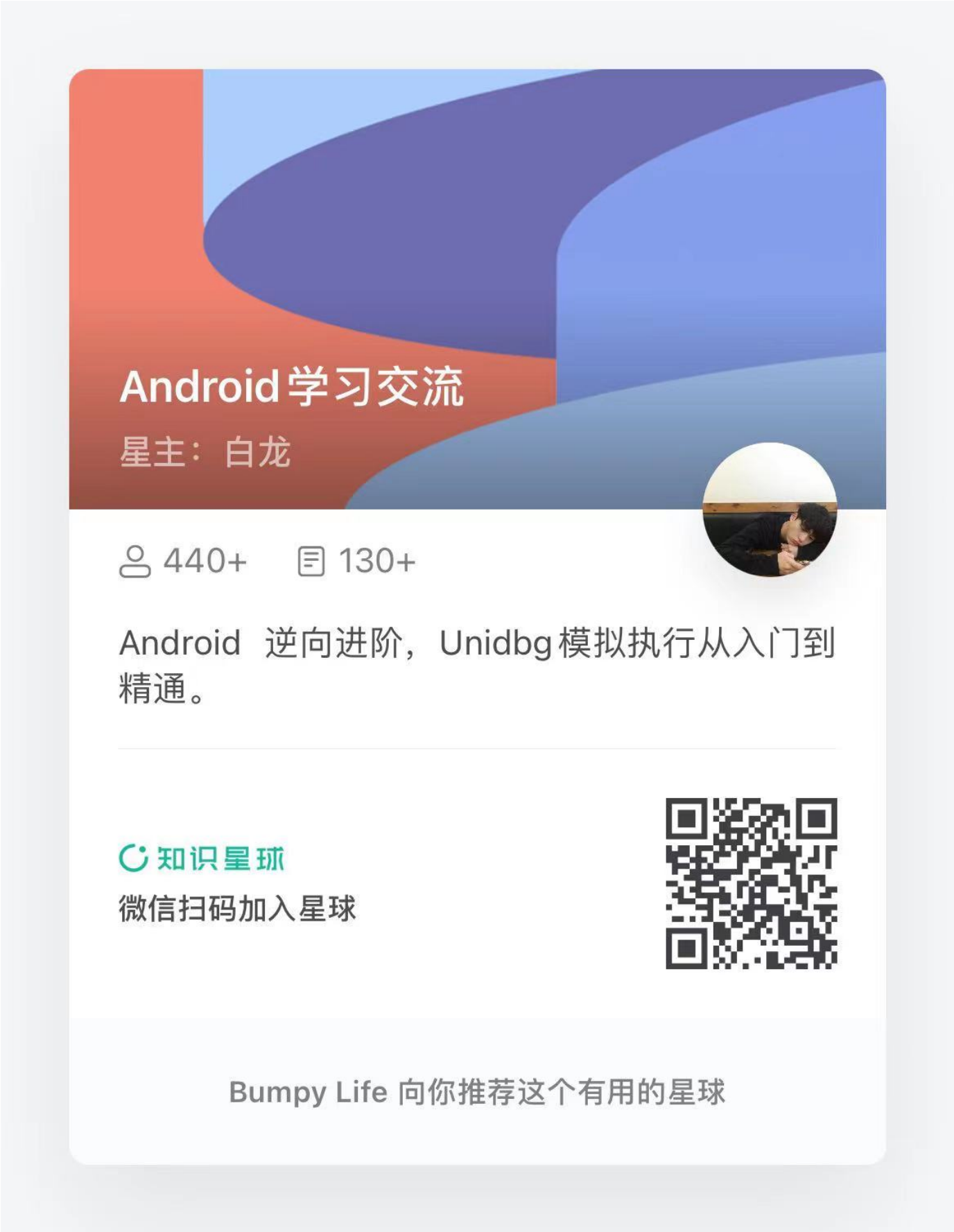
【转载】第七讲——使用Unidbg进行DFA攻击 (https://www.qinless.com/1659)

【转载】第八讲——案例 (https://www.qinless.com/1661)

« 上一篇文章 (https://www.qinless.com/1655)
下一篇文章 (https://www.qinless.com/1659) »

说说在前面

本文转载龙哥星球 白盒 AES 密码学系列，相关文件代码等，可加入星球下载



(https://www.qinless.com/wp-content/uploads/2022/03/longge-zsxq.jpeg)

一、前言

这篇里，我们展示使用Frida处理DFA，这是更常见的场景。上篇文章那般拥有源码的情况下，注入故障就是修改代码，而一般的逆向场景中，我们需要IDA分析代码以及使用Frida API 或IDA 修改 state的字节。

二、案例

首先，这个样例没做符号的隐藏，cryptoDD.so的功能一目了然。这是半年前的版本，在样本新版中不存在这样的信息泄露。

```
LOAD:00000000
LOAD:00000000 ; File Name   : [illegible] libcryptoDD.so
LOAD:00000000 ; Format      : ELF for ARM (Shared object)
LOAD:00000000 ; Needed Library 'libandroid.so'
LOAD:00000000 ; Needed Library 'liblog.so'
LOAD:00000000 ; Needed Library 'libm.so'
LOAD:00000000 ; Needed Library 'libdl.so'
LOAD:00000000 ; Needed Library 'libc.so'
LOAD:00000000 ; Shared Name 'libcryptoDD.so'
LOAD:00000000 ;
LOAD:00000000 ; Options     : EF_ARM_SOFT_FLOAT
LOAD:00000000 ; EABI version: 5
LOAD:00000000 ;
LOAD:00000000 ; Source File : 'crtbegin_so.c'
LOAD:00000000 ; Source File : 'D:\AndroidProjects\safebox-dev-whitebox\safeboxlib\src\main\cpp\algorithm\AES.c'
LOAD:00000000 ; Source File : 'D:\AndroidProjects\safebox-dev-whitebox\safeboxlib\src\main\cpp\algorithm\RSA\RSA.c'
LOAD:00000000 ; Source File : 'D:\AndroidProjects\safebox-dev-whitebox\safeboxlib\src\main\cpp\algorithm\RSA\prime.c'
LOAD:00000000 ; Source File : 'D:\AndroidProjects\safebox-dev-whitebox\safeboxlib\src\main\cpp\algorithm\RSA\bignum.c'
LOAD:00000000 ; Source File : 'D:\AndroidProjects\safebox-dev-whitebox\safeboxlib\src\main\cpp\algorithm\padding.c'
LOAD:00000000 ; Source File : 'D:\AndroidProjects\safebox-dev-whitebox\safeboxlib\src\main\cpp\algorithm\base64.c'
LOAD:00000000 ; Source File : 'D:\AndroidProjects\safebox-dev-whitebox\safeboxlib\src\main\cpp\algorithm\md5.c'
LOAD:00000000 ; Source File : 'D:\AndroidProjects\safebox-dev-whitebox\safeboxlib\src\main\cpp\algorithm\WBAES.c'
LOAD:00000000 ; Source File : 'D:\AndroidProjects\safebox-dev-whitebox\safeboxlib\src\main\cpp\Main.c'
LOAD:00000000 ; Source File : 'bpabi.c'
LOAD:00000000 ; Source File : 'unwind-arm.c'
LOAD:00000000 ; Source File : 'pr-support.c'
LOAD:00000000 ; Source File : 'libgcc2.c'
LOAD:00000000
```

qinless.com

(https://www.qinless.com/wp-content/uploads/2022/04/62470223b0e76.png)

从源文件信息可知，SO中有AES、RSA、Base64、MD5以及WBAES。我们关注的是WBAES，因为这名字一般是WhiteBox AES即白盒AES的缩写。

Function name

- j_android_native_wbaes
- wbShiftRows
- wbInvShiftRows
- aes128_enc_wb_coff
- aes128_enc_wb_xlc
- aes128_dec_wb_coff
- aes128_dec_wb_xlc
- wbaes_encrypt_ecb
- wbaes_decrypt_ecb
- android_native_wbaes_jni
- android_native_wbaes

qinless.com

wb

(https://www.qinless.com/wp-content/uploads/2022/04/6247023260053.png)

(https://www.qiniess.com/wp-content/uploads/2022/04/6247023260033.png)

我打算对wbaes_encrypt_ecb 函数做分析，SO中同样没去掉这些符号名，感恩。

```
int32_t __fastcall wbaes_encrypt_ecb(const uint8_t *in, uint32_t in_len, uint8_t *out, uint32_t
```

可以看到，参数1是入参，参数2是入参的长度，参数3存放结果，参数4疑似是某种模式。

用Frida Hook 这个函数，查看调用情况，attach 模式遇到了如下报错

> Failed to attach: unable to access process with pid 12225 due to system restrictions; try `sudo sysctl kernel.yama.ptrace_scope=0`, or run Frida as root

为什么attach失败？attach 需要使用ptrace实现注入，如果样本存在双进程保护，ptrace被占用无法使用，就会导致attach注入失败。

看一下样本是否存在双进程保护

```
C:\Users\13352>adb shell
polaris:/ $ su
1|polaris:/ # ps -A | grep lucky
u0_a299      14188   689 1935344 411676 SyS_epoll_wait      0 S com.lucky.luckyclient
u0_a299      14238 14188 1489108  93556 __skb_wait_for_more_packets 0 S com.lucky.luckyclient
polaris:/ #
```

确实有，改用Frida Spawn 就OK了。代码如下

```
function hookWBAES(){
    var base_addr = Module.findBaseAddress("libcryptoDD.so");
    // wbaes_encrypt_ecb 的偏移地址，因为thumb所以+1
    var real_addr = base_addr.add(0x17bd5)
    Interceptor.attach(real_addr, {
        onEnter: function (args) {
            console.log("Enter");
        }
    });
}
```

命令：**frida -U -f com.lucky.luckyclient -l hookaes.js --no-pause**

需要注意的是，我并没有在代码中调用hookWBAES函数，而是Spawn打开App后，在交互命令行中调用它。后面所有的Frida注入均采用这个方式调用函数。新手可能会困惑，我们做的原因在于，spawn的时机点极早，libcryptoDD.so在那个时机一定还未加载，findBaseAddress自然一无所获，没有办法实现Hook。所以等App打开，SO加载完毕后的时机再调用函数，才能顺利Hook。这么做的缺点也很明显，如果目标函数只在SO加载的较早时机被执行一次，那么我们较晚的时机就没法做观测和拦截，应对这种情况，需要Hook dlopen 或更早的call_function以确保更早的Hook时机。好在我们的函数在稍晚一些的时机仍然有被调用，不必考虑时机的问题。

COPY

```
     ____
    / _  |   Frida 14.2.18 - A world-class dynamic instrumentation toolkit
   | (_| |
    > _  |   Commands:
   /_/ |_|       help      -> Displays the help system
   . . . .       object?   -> Display information about 'object'
   . . . .       exit/quit -> Exit
   . . . .
   . . . .   More info at https://frida.re/docs/home/
Spawned `com.lucky.luckyclient`. Resuming main thread!
[MIX 2S::com.lucky.luckyclient]-> hookWBAES()
[MIX 2S::com.lucky.luckyclient]-> Enter
Enter
Enter
Enter
Enter
```

确认这个函数有被执行后，我们做更详细的Hook

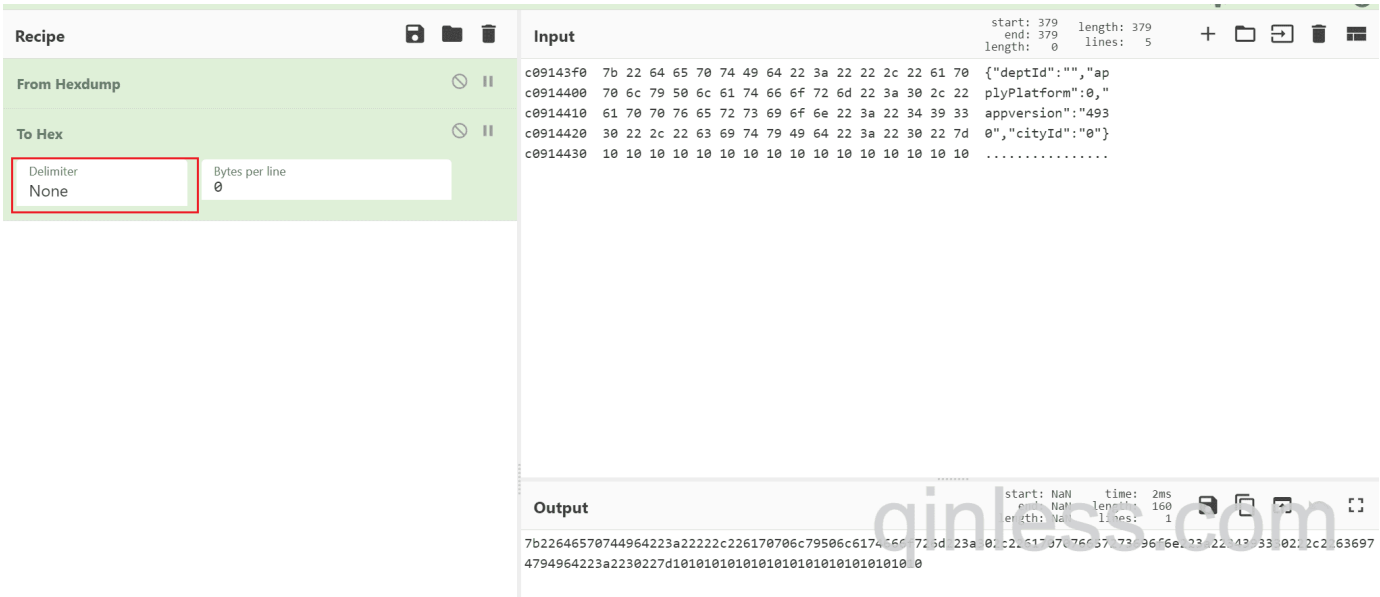
```
function hookWBAES(){
    var base_addr = Module.findBaseAddress("libcryptoDD.so");
    var real_addr = base_addr.add(0x17bd5)
    Interceptor.attach(real_addr, {
        onEnter: function (args) {
            console.log("Input");
            this.buffer = args[2];
            this.length = args[1].toInt32();
            console.log(hexdump(args[0],{length: this.length}));
            console.log("mode:"+args[3]);
        },
        onLeave: function (retval) {
            console.log("Output")
            console.log(hexdump(this.buffer,{length: this.length}));
        }
    });
}
```

在函数进入前，以in_len值为长度，打印待加密的内存块，并打印参数4；在函数结束后，等长度打印存放结果的内存块。因为我们知道，AES加密的输入输出等长。随便截取一次结果进行分析

```
Input
           0  1  2  3  4  5  6  7  8  9  A  B  C  D  E  F  0123456789ABCDEF
c09143f0  7b 22 64 65 70 74 49 64 22 3a 22 22 2c 22 61 70  {"deptId":"","ap
c0914400  70 6c 79 50 6c 61 74 66 6f 72 6d 22 3a 30 2c 22  plyPlatform":0,"
c0914410  61 70 70 76 65 72 73 69 6f 6e 22 3a 22 34 39 33  appversion":"493
c0914420  30 22 2c 22 63 69 74 79 49 64 22 3a 22 30 22 7d  0","cityId":"0"}
c0914430  10 10 10 10 10 10 10 10 10 10 10 10 10 10 10 10  ................
mode:0x0
Output
           0  1  2  3  4  5  6  7  8  9  A  B  C  D  E  F  0123456789ABCDEF
c0912e60  15 55 3f 75 b1 2b 7f df c6 95 6e c9 55 66 dd 52  .U?u.+....n.Uf.R
c0912e70  4c 60 09 d9 22 f8 b3 cd 8b 65 f8 4a 8b fd 60 01  L`.."....e.J..`.
c0912e80  48 13 8b e4 7f fd 3c c4 13 93 a3 ba db 3c d5 0b  H.....<......<..
c0912e90  0d 77 f3 5b 7f 3d b1 f0 8c d4 0a 3f 47 66 83 51  .w.[.=.....?Gf.Q
c0912ea0  15 da 7c 4c 92 3d 63 d1 12 c3 88 11 91 6b 41 62  ..|L.=c......kAb
```

观察输入可以发现，待加密的内容经过了PCKS7填充，末尾的一整行0x10就是证据。

接下来我们使用Frida Call复现函数调用，看能否得到相同结果。首先我在 [Cyberchef](https://gchq.github.io/CyberChef/#recipe=From_Hexdump()To_Hex('None',0)&input=YzA5MTQzZjAgIDdiIDIyIDY0IDY1IDcwIDc0IDQ5IDY0IDIyIDNhIDIyIDIyIDJjIDIyIDYxIDcwICB7ImRlcHRJZCI6IiIsImFwCmMwOTE0NDAwICA3MCA2YyA3OSA1MCA2YyA2MSA3NCA2NiA2ZiA3MiA2ZCAyMiAzYSAzMCAyYyAyMiAgcGx5UGxhdGZvcm0iOjAsIgpjMDkxNDQxMCAgNjEgNzAgNzAgNzYgNjUgNzIgNzMgNjkgNmYgNmUgMjIgM2EgMjIgMzQgMzkgMzMgIGFwcHZlcnNpb24iOiI0OTMKYzA5MTQ0MjAgIDMwIDIyIDJjIDIyIDYzIDY5IDc0IDc5IDQ5IDY0IDIyIDNhIDIyIDMwIDIyIDdkICAwIiwiY2l0eUlkIjoiMCJ9CmMwOTE0NDMwICAxMCAxMCAxMCAxMCAxMCAxMCAxMCAxMCAxMCAxMCAxMCAxMCAxMCAxMCAxMCAxMCAxMCAxMCAxMCAxMCAxMCAgLi4uLi4uLi4uLi4uLg
(https://gchq.github.io/CyberChef/#recipe=From_Hexdump()To_Hex('None',0)&input=YzA5MTQzZjAgIDdiIDIyIDY0IDY1IDcwIDc0IDQ5IDY0IDIyIDNhIDIyIDIyIDJjIDIyIDYxIDcwICB7ImRlcHRJZCI6IiIsImFwCmMwOTE0NDAwICA3MCA2YyA3OSA1MCA2YyA2MSA3NCA2NiA2ZiA3MiA2ZCAyMiAzYSAzMCAyYyAyMiAgcGx5UGxhdGZvcm0iOjAsIgpjMDkxNDQxMCAgNjEgNzAgNzAgNzYgNjUgNzIgNzMgNjkgNmYgNmUgMjIgM2EgMjIgMzQgMzkgMzMgIGFwcHZlcnNpb24iOiI0OTMKYzA5MTQ0MjAgIDMwIDIyIDJjIDIyIDYzIDY5IDc0IDc5IDQ5IDY0IDIyIDNhIDIyIDMwIDIyIDdkICAwIiwiY2l0eUlkIjoiMCJ9CmMwOTE0NDMwICAxMCAxMCAxMCAxMCAxMCAxMCAxMCAxMCAxMCAxMCAxMCAxMCAxMCAxMCAxMCAxMCAxMCAxMCAxMCAxMCAxMCAgLi4uLi4uLi4uLi4uLg)) 中将hexdump结果转成hexstring

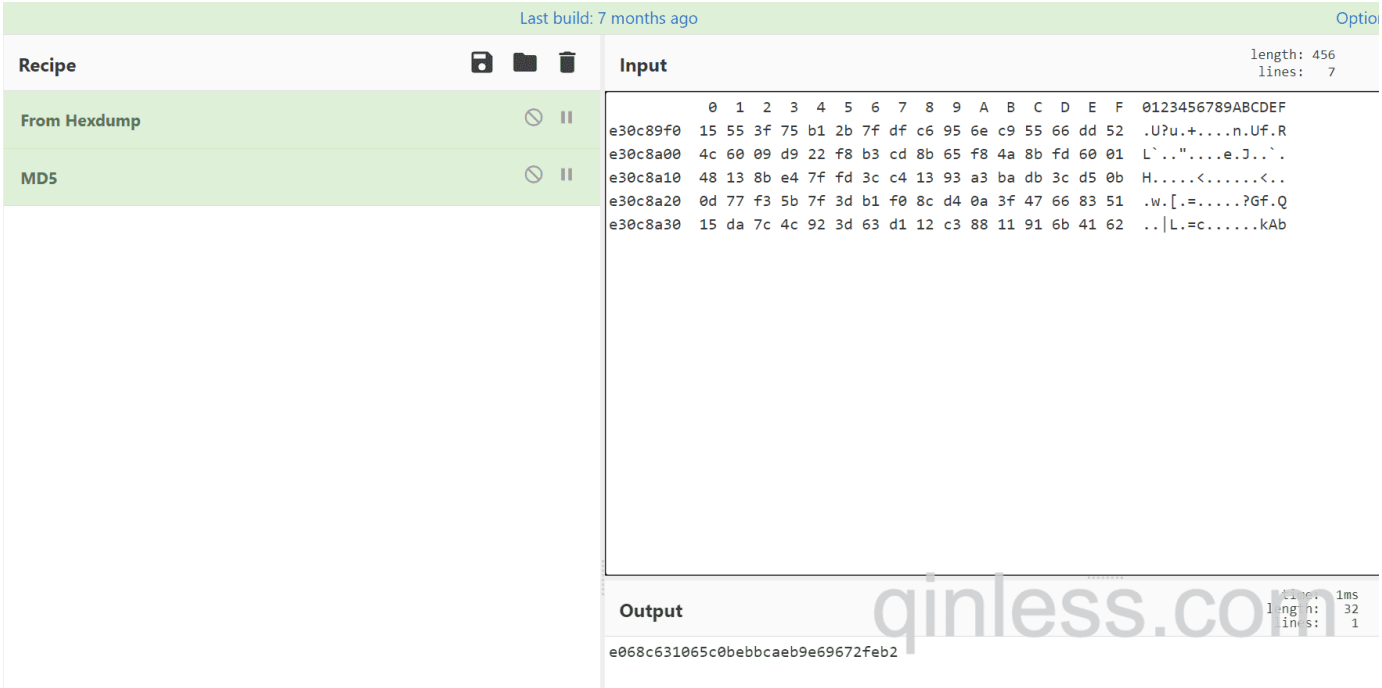


(https://www.qinless.com/wp-content/uploads/2022/04/624702436fa63.png)

下面是Frida Call 脚本，需要注意的是，为了防止我们的Call与App自己的逻辑混在一起，我选择在Frida Spawn 调起App 数秒后，将手机息屏，然后再调用我们Frida脚本中相关函数。后面所有Call的地方均需要使用这个技巧。

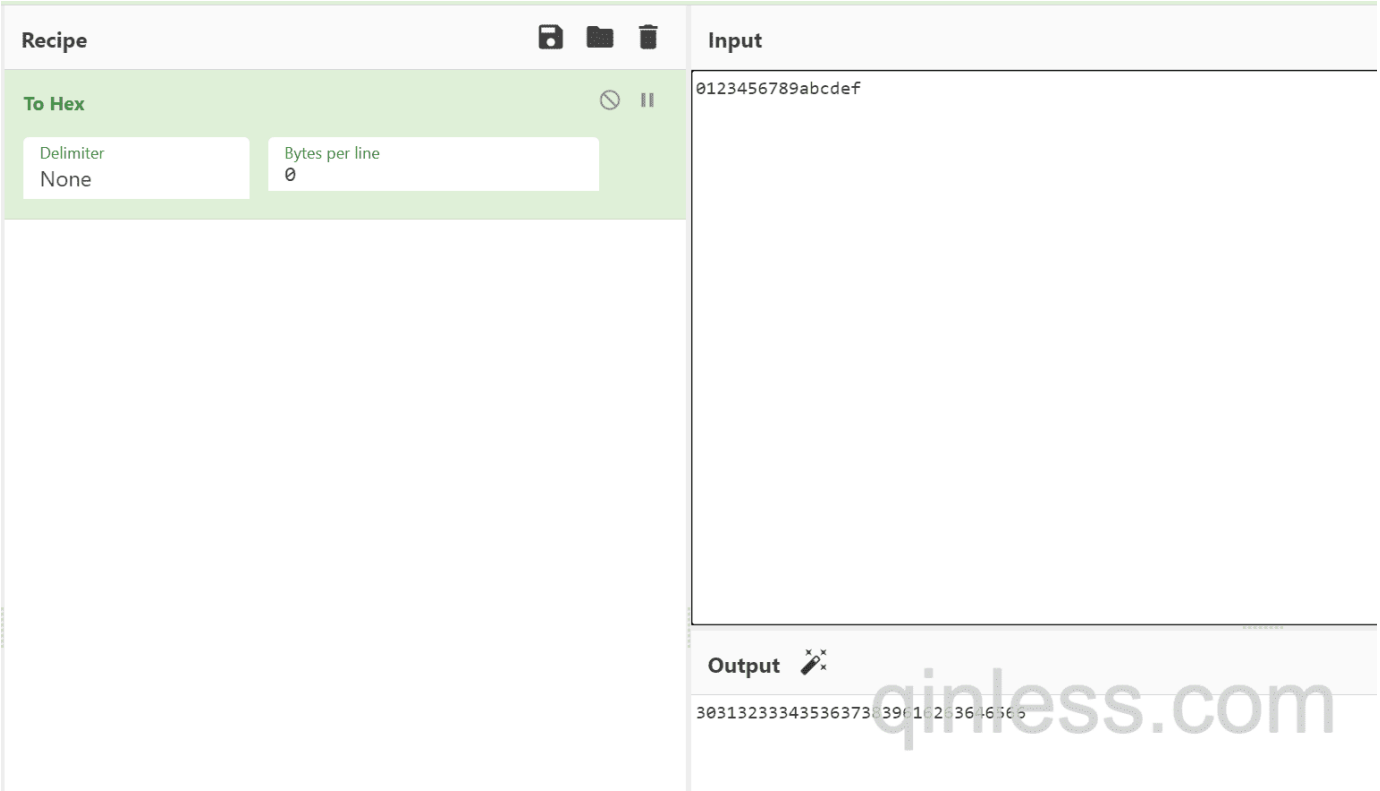
```
function hexToBytes(hex) {
    for (var bytes = [], c = 0; c < hex.length; c += 2)
        bytes.push(parseInt(hex.substr(c, 2), 16));
    return bytes;
}

function callAES(){
    var base_addr = Module.findBaseAddress("libcryptoDD.so");
    var real_addr = base_addr.add(0x17bd5);
    var wbaes_encrypt_ecb_func = new NativeFunction(real_addr, "int", ["pointer", "int", "point
    var inputPtr = Memory.alloc(0x50);
    var inputArray = hexToBytes("7b22646570744964223a22222c226170706c79506c6174666f726d223a302c
    Memory.writeByteArray(inputPtr, inputArray)
    var outputPtr = Memory.alloc(0x50);
    wbaes_encrypt_ecb_func(inputPtr, 0x50, outputPtr, 0);
    console.log(hexdump(outputPtr,{length: 0x50}));
}
```

Frida Call的结果看着是一样的，不放心的可以像我一样，比对两者的MD5结果。



(https://www.qinless.com/wp-content/uploads/2022/04/6247024fe4017.png)

下面做什么好呢？我想测试一下它是不是真的ECB模式，万一样本虚晃一枪，实际上是CBC模式呢。因为样本经过一定程度的混淆，静态看代码确实可能看走眼。



(https://www.qinless.com/wp-content/uploads/2022/04/6247025ae44d8.png)

我决定输入两组“0123456789abcdef”

```
function hexToBytes(hex) {
    for (var bytes = [], c = 0; c < hex.length; c += 2)
        bytes.push(parseInt(hex.substr(c, 2), 16));
    return bytes;
}

function callAES(){
    var base_addr = Module.findBaseAddress("libcryptoDD.so");
    var real_addr = base_addr.add(0x17bd5);
    var wbaes_encrypt_ecb_func = new NativeFunction(real_addr, "int", ["pointer", "int", "point
    var inputPtr = Memory.alloc(0x20);
    var inputArray = hexToBytes("30313233343536373839616263646566303132333435363738396162636465
    Memory.writeByteArray(inputPtr, inputArray)
    var outputPtr = Memory.alloc(0x20);
    wbaes_encrypt_ecb_func(inputPtr, 0x20, outputPtr, 0);
    console.log(hexdump(outputPtr,{length: 0x20}));
}
```

看一下执行情况

```
Spawned `com.lucky.luckyclient`. Resuming main thread!
[MIX 2S::com.lucky.luckyclient]-> callAES()
           0  1  2  3  4  5  6  7  8  9  A  B  C  D  E  F  0123456789ABCDEF
e2ba4c90  b0 f5 9c 0d 48 c1 45 91 5f c8 f6 a8 42 c4 d5 eb  ....H.E._...B...
e2ba4ca0  b0 f5 9c 0d 48 c1 45 91 5f c8 f6 a8 42 c4 d5 eb  ....H.E._...B...
[MIX 2S::com.lucky.luckyclient]->
```

可以发现，两个分组加密结果一致，看来确实是ECB。

接下来考虑DFA攻击，就像前面做的一样，我们需要关注于单个分组，最后的call代码如下

```
function hexToBytes(hex) {
    for (var bytes = [], c = 0; c < hex.length; c += 2)
        bytes.push(parseInt(hex.substr(c, 2), 16));
    return bytes;
}

function callAES(){
    var base_addr = Module.findBaseAddress("libcryptoDD.so");
    var real_addr = base_addr.add(0x17bd5);
    var wbaes_encrypt_ecb_func = new NativeFunction(real_addr, "int", ["pointer", "int", "point
    var inputPtr = Memory.alloc(0x10);
    var inputArray = hexToBytes("303132333435363738396162636465 66");
    Memory.writeByteArray(inputPtr, inputArray)
    var outputPtr = Memory.alloc(0x10);
    wbaes_encrypt_ecb_func(inputPtr, 0x10, outputPtr, 0);
    console.log(hexdump(outputPtr,{length: 0x10}));
}
```

接下来看函数的代码情况，尽管存在一些混淆，但是也能看出，wbaes_encrypt_ecb只是外层的简单包装，aes128_enc_wb_coff 和 aes128_enc_wb_xlc 才是归宿。

符号没去掉给了我们很大的帮助

```
void __fastcall aes128_enc_wb_xlc(uint8_t *in, uint8_t *out);
void __fastcall aes128_enc_wb_coff(uint8_t *in, uint8_t *out)
```

完善Hook脚本，添加对这两个函数的Hook

COPY

```
function hexToBytes(hex) {
    for (var bytes = [], c = 0; c < hex.length; c += 2)
        bytes.push(parseInt(hex.substr(c, 2), 16));
    return bytes;
}

function hookXlc(){
    var base_addr = Module.findBaseAddress("libcryptoDD.so");
    // aes128_enc_wb_xlc 的偏移地址，因为thumb所以+1
    var real_addr = base_addr.add(0x15C8D)
    Interceptor.attach(real_addr, {
        onEnter: function (args) {
            console.log("aes128_enc_wb_xlc");
            console.log(hexdump(args[0],{length: 0x10}))
        }
    });
}

function hookCoff(){
    var base_addr = Module.findBaseAddress("libcryptoDD.so");
    // aes128_enc_wb_coff 的偏移地址，因为thumb所以+1
    var real_addr = base_addr.add(0x15321)
    Interceptor.attach(real_addr, {
        onEnter: function (args) {
            console.log("aes128_enc_wb_coff");
            console.log(hexdump(args[0],{length: 0x10}))
        }
    });
}

function callAES(){
    var base_addr = Module.findBaseAddress("libcryptoDD.so");
    var real_addr = base_addr.add(0x17bd5);
    var wbaes_encrypt_ecb_func = new NativeFunction(real_addr, "int", ["pointer", "int", "point
    var inputPtr = Memory.alloc(0x10);
    var inputArray = hexToBytes("303132333435363738396162636465 66");
    Memory.writeByteArray(inputPtr, inputArray)
    var outputPtr = Memory.alloc(0x10);
    wbaes_encrypt_ecb_func(inputPtr, 0x10, outputPtr, 0);
    console.log(hexdump(outputPtr,{length: 0x10}));
}
```

执行情况如下

```
[MIX 2S::com.lucky.luckyclient]-> hookCoff()
[MIX 2S::com.lucky.luckyclient]-> hookXlc()
[MIX 2S::com.lucky.luckyclient]-> callAES()
aes128_enc_wb_coff
          0  1  2  3  4  5  6  7  8  9  A  B  C  D  E  F  0123456789ABCDEF
c45bcc78  30 31 32 33 34 35 36 37 38 39 61 62 63 64 65 66  0123456789abcdef
          0  1  2  3  4  5  6  7  8  9  A  B  C  D  E  F  0123456789ABCDEF
e315efe8  b0 f5 9c 0d 48 c1 45 91 5f c8 f6 a8 42 c4 d5 eb  ....H.E._...B...
```

可以确认，我们的逻辑其实最终走到了aes128_enc_wb_coff，下面分析aes128_enc_wb_coff。观察一番可以看到三个表

```
const uint8_t Tboxes_[16][256];
const uint8_t Txor[16][16];
const uint32_t Tyboxes[9][16][256];
```

都是不小的表，且程序逻辑中存在大量查表运算，确实是白盒加密。

按照上一篇中总结的经验，接下来是三步骤

- 找轮
- 找时机，即具体第几轮做故障注入
- 找state

由于函数被混淆了，很难静态做出这些判断。一个可行的办法是分析代码，寻找锚点。

我们找一下锚点，我们发现样本中存在对wbShiftRows这个函数的调用。在上一篇文章，以及更前面对标准算法的学习中，我们知道，循环左移是一轮运算中四步骤之一，而一次加密中，存在十轮运算。这个wbShiftRows就是锚点，下面增加对它的Hook

COPY

```
function hexToBytes(hex) {
    for (var bytes = [], c = 0; c < hex.length; c += 2)
        bytes.push(parseInt(hex.substr(c, 2), 16));
    return bytes;
}

function hookXlc(){
    var base_addr = Module.findBaseAddress("libcryptoDD.so");
    // aes128_enc_wb_xlc 的偏移地址，因为thumb所以+1
    var real_addr = base_addr.add(0x15C8D)
    Interceptor.attach(real_addr, {
        onEnter: function (args) {
            console.log("aes128_enc_wb_xlc");
            console.log(hexdump(args[0],{length: 0x10}))
        }
    });
}

function hookCoff(){
    var base_addr = Module.findBaseAddress("libcryptoDD.so");
    // aes128_enc_wb_coff 的偏移地址，因为thumb所以+1
    var real_addr = base_addr.add(0x15321)
    Interceptor.attach(real_addr, {
        onEnter: function (args) {
            console.log("aes128_enc_wb_coff");
            console.log(hexdump(args[0],{length: 0x10}))
        }
    });
}

function hookwbShiftRows(){
    var count = 0;
    var base_addr = Module.findBaseAddress("libcryptoDD.so");
    // wbShiftRows 的偏移地址，因为thumb所以+1
    var real_addr = base_addr.add(0x14F99)
    Interceptor.attach(real_addr, {
        onEnter: function (args) {
            count += 1;
            console.log("wbShiftRows:"+count);
        }
    });
}
```

```
function callAES(){
    var base_addr = Module.findBaseAddress("libcryptoDD.so");
    var real_addr = base_addr.add(0x17bd5);
    var wbaes_encrypt_ecb_func = new NativeFunction(real_addr, "int", ["pointer", "int", "point
    var inputPtr = Memory.alloc(0x10);
    var inputArray = hexToBytes("303132333435363738396162636465 66");
    Memory.writeByteArray(inputPtr, inputArray)
    var outputPtr = Memory.alloc(0x10);
    wbaes_encrypt_ecb_func(inputPtr, 0x10, outputPtr, 0);
    console.log(hexdump(outputPtr,{length: 0x10}));
}
```

运行效果如下

```
[MIX 2S::com.lucky.luckyclient]-> hookCoff()
[MIX 2S::com.lucky.luckyclient]-> hookXlc()
[MIX 2S::com.lucky.luckyclient]-> hookwbShiftRows()
[MIX 2S::com.lucky.luckyclient]-> callAES()
aes128_enc_wb_coff
           0  1  2  3  4  5  6  7  8  9  A  B  C  D  E  F  0123456789ABCDEF
c45bcc78  30 31 32 33 34 35 36 37 38 39 61 62 63 64 65 66  0123456789abcdef
wbShiftRows:1
wbShiftRows:2
wbShiftRows:3
wbShiftRows:4
wbShiftRows:5
wbShiftRows:6
wbShiftRows:7
wbShiftRows:8
wbShiftRows:9
wbShiftRows:10
           0  1  2  3  4  5  6  7  8  9  A  B  C  D  E  F  0123456789ABCDEF
e2b41258  b0 f5 9c 0d 48 c1 45 91 5f c8 f6 a8 42 c4 d5 eb  ....H.E._...B...
```

我们惊喜的发现，wbShiftRows确实是十轮，那么按照前文所学，第九轮的起始处是一个好时机。那么state呢？在标准实现下，循环左移作用于state，那么wbShiftRows的入参是不是也一样是 $state$ ？可以做这样的猜测，下面看代码分析。

```
void __fastcall wbShiftRows(uint8_t *out)
```

可以发现，函数就一个参数，out即作输入又当输出。那么我们修改这个入参，就修改了state。除了”锚点论“以外，在后续Unidbg处理DFA的相关篇幅中，我们会介绍另外一种办法。

故障注入如下，我将第一个字节改成0。下面仅贴修改部分的代码

```
function hookwbShiftRows(){
    var count = 0;
    var base_addr = Module.findBaseAddress("libcryptoDD.so");
    // wbShiftRows 的偏移地址，因为thumb所以+1
    var real_addr = base_addr.add(0x14F99)
    Interceptor.attach(real_addr, {
        onEnter: function (args) {
            count += 1;
            if(count===9){
                args[0].writeS8(0x0);
            }
            console.log("wbShiftRows:"+count);
        }
    });
}
```

运行情况如下

```
PS C:\Users\13352\Desktop\白盒密码学\星球连载\星球系列\第六讲——使用Frida进行DFA攻击\files\案例
     ____
    / _  |   Frida 14.2.18 - A world-class dynamic instrumentation toolkit
   | (_| |
    > _  |   Commands:
   /_/ |_|       help      -> Displays the help system
   . . . .       object?   -> Display information about 'object'
   . . . .       exit/quit -> Exit
   . . . .
   . . . .   More info at https://frida.re/docs/home/
Spawned `com.lucky.luckyclient`. Resuming main thread!
[MIX 2S::com.lucky.luckyclient]->
[MIX 2S::com.lucky.luckyclient]->
[MIX 2S::com.lucky.luckyclient]-> hookwbShiftRows()
[MIX 2S::com.lucky.luckyclient]-> callAES()
wbShiftRows:1
wbShiftRows:2
wbShiftRows:3
wbShiftRows:4
wbShiftRows:5
wbShiftRows:6
wbShiftRows:7
wbShiftRows:8
wbShiftRows:9
wbShiftRows:10
           0  1  2  3  4  5  6  7  8  9  A  B  C  D  E  F  0123456789ABCDEF
e2baa298  2d f5 9c 0d 48 c1 45 eb 5f c8 c0 a8 42 78 d5 eb  -...H.E._...Bx..
```

将故障密文和正常密文做对比

正常密文 b0 f5 9c 0d 48 c1 45 91 5f c8 f6 a8 42 c4 d5 eb

故障密文 2d f5 9c 0d 48 c1 45 eb 5f c8 c0 a8 42 78 d5 eb

可以发现，这是加密情况下，故障成功注入的四种模式之一，非常好，我们上面的猜测都对。

接下来收集正常密文，以及故障密文，为了方便多次注入，以及数据方便的喂给phoenixAES，代码做了一些调整。

COPY

```
function hexToBytes(hex) {
    for (var bytes = [], c = 0; c < hex.length; c += 2)
        bytes.push(parseInt(hex.substr(c, 2), 16));
    return bytes;
}

function hookXlc(){
    var base_addr = Module.findBaseAddress("libcryptoDD.so");
    // aes128_enc_wb_xlc 的偏移地址，因为thumb所以+1
    var real_addr = base_addr.add(0x15C8D)
    Interceptor.attach(real_addr, {
        onEnter: function (args) {
            console.log("aes128_enc_wb_xlc");
            console.log(hexdump(args[0],{length: 0x10}))
        }
    });
}

function hookCoff(){
    var base_addr = Module.findBaseAddress("libcryptoDD.so");
    // aes128_enc_wb_coff 的偏移地址，因为thumb所以+1
    var real_addr = base_addr.add(0x15321)
    Interceptor.attach(real_addr, {
        onEnter: function (args) {
            console.log("aes128_enc_wb_coff");
            console.log(hexdump(args[0],{length: 0x10}))
        }
    });
}

//生成从minNum到maxNum的随机数
function randomNum(minNum,maxNum){
    if (arguments.length === 1) {
        return parseInt(Math.random() * minNum + 1, 10);
    } else if (arguments.length === 2) {
        return parseInt(Math.random() * (maxNum - minNum + 1) + minNum, 10);
    } else {
        return 0;
    }
}

function hookwbShiftRows(){
    var count = 0;
```

```
    var base_addr = Module.findBaseAddress("libcryptoDD.so");
    // wbShiftRows 的偏移地址，因为thumb所以+1
    var real_addr = base_addr.add(0x14F99)
    Interceptor.attach(real_addr, {
        onEnter: function (args) {
            count += 1;
            if(count%9===0){
                args[0].add(randomNum(0,15)).writeS8(randomNum(0, 0xff));
            }
        }
    });
}

function callAES(){
    var base_addr = Module.findBaseAddress("libcryptoDD.so");
    var real_addr = base_addr.add(0x17bd5);
    var wbaes_encrypt_ecb_func = new NativeFunction(real_addr, "int", ["pointer", "int", "point
    var inputPtr = Memory.alloc(0x10);
    var inputArray = hexToBytes("303132333435363738396162636465 66");
    Memory.writeByteArray(inputPtr, inputArray)
    var outputPtr = Memory.alloc(0x10);
    wbaes_encrypt_ecb_func(inputPtr, 0x10, outputPtr, 0);
    var output = Memory.readByteArray(outputPtr, 0x10);
    console.log(bufferToHex(output))

}

function bufferToHex (buffer) {
    return [...new Uint8Array (buffer)]
        .map (b => b.toString (16).padStart (2, "0"))
        .join ("");
}

function dfa(){
    hookwbShiftRows();
    for(var i=0;i<200;i++){
        callAES();
    }
}
```

- dfa函数中，两百次调用callAES，避免一次次手动注入故障。
- callAES 函数在打印结果时，使用hexString 而不是 hexdump。

- hookwbShiftRows 中，计数器为九的倍数时就注入故障，因为计数器的数字会随着callAES不断增加。

将正确密文以及输出的200个故障密文放入phoenixAES，顺利出结果。

```
Round key bytes recovered:
869D92BBB700D0D25BD9FD3E224B5DF2
```

放入 stark

```
C:\Users\13352\CLionProjects\mystark\cmake-build-debug>mystark.exe 869D92BBB700D0D25BD9FD3E224B
K00: 644A4C64434A69566E44764D394A5570
K01: B3B61D76F0FC74209EB8026DA7F2571D
K02: 38EDB92AC811CD0A56A9CF67F15B987A
K03: 05AB638BCDBAAE819B1361E66A48F99C
K04: 5F32BD8992881308099B72EE63D38B72
K05: 290FFD72BB87EE7AB21C9C94D1CF17E6
K06: 83FF734C38789D368A6401A25BAB1644
K07: A1B8687599C0F54313A4F4E1480FE2A5
K08: 57206E27CEE09B64DD446F85954B8D20
K09: FF7DD90D319D4269ECD92DEC7992A0CC
K10: 869D92BBB700D0D25BD9FD3E224B5DF2
```

得出密钥：**644A4C64434A69566E44764D394A5570**，在pureAes128Encrypt.py中做验证，完全正确。

三、关于自动化的思考

自动化是美丽且诱人的，找轮/找时机/找state 这三个步骤能否自动化实现。目前学术研究上认为，有两个路径可以达成其自动化，1是深度学习 2是暴力遍历。我并不关注第一条路径，深度学习的水太深了，很难把握住。第二条路径的可行性以及可解释性都更高。

暴力遍历的思路很简单，在程序执行流中随机制造一个错误，利用计算机的高性能进行大量试错，最后把几十上万条fault结果放到类似phoenixAES这样的工具里，由其筛选出可用的故障密文，运算和约束出Key的结果。

所谓在执行流中随机制造一个错误，有如下可能的手段

- 白盒化基于查表，那么随机修改其所依赖的表中的一字节
- 在执行流中随机跳过某一条汇编指令的执行，即动态nop一条
- 监控内存读写，随机失败一次

这些故障均存在一定的可能性，导致在合适的轮、合适的时机，对state 修改了一个字节的实质效果。但这个可能性有多大？恐怕需要深思。尤其考虑到目前样本广泛且普遍存在的混淆和防护，我认为在Android SO上做此类自动化工作性价比并不高，还是老老实实找特征来得方便。

但还是要说，如果有读者试图这么做，建议不要用phoenixAES，而是用C/C++复写逻辑，相较于Python实现可以快十数倍。

四、尾声

感兴趣的可以将 mode参数 改成2或其他值，可能会导致新的逻辑以及新的密钥，可以探索嗷。

标签: # aes, # dfa, # frida, # 攻击模型, # 白盒aes (https://www.qinless.com/tag/baiheaes)

(https://www.qinless.com/1657)

(https://www.qinless.com/wp-content/themes/honey/public/qrcode?data=https://www.qinless.com/1657)

(https://www.qinless.com/1657) 暂无评论 (https://www.qinless.com/1657#respond)

【转载】第五讲——使用源码进行DFA攻击
上一篇
(https://www.qinless.com/1655)

【转载】第七讲——使用Unidbg进行DFA攻击
下一篇
(https://www.qinless.com/1659)

会爬山的小脑虎 (https://www.qinless.com/author/xiayu)
小爬爬

本文作者： 会爬山的小脑虎 (https://www.qinless.com/author/xiayu)
本文链接： https://www.qinless.com/?p=1657 (https://www.qinless.com/1657)
版权声明：本博客所有文章除特别声明外，均采用 CC BY-NC-SA 4.0 (https://creativecommons.org/licenses/by-nc-sa/4.0/) **许可协议。转载请注明来自** qinless 的博客！ (https://www.qinless.com)

微信搜一搜

逆向小友

发表评论

昵称 *

邮箱 *

网址

输入评论内容...

提交评论

Copyright 2022. All Rights Reserved. Powered By XinTheme (http://www.xintheme.com) + WordPress 果酱 (https://blog.wpjam.com/) 浙ICP备2021029793号 (http://www.miibeian.gov.cn/)